

Dynamic Software Updating and State Migration Protocols for Real-Time DSP Dataflow Graphs: A Compiler Runtime and Architectural Analysis

Dynamic Software Updating (DSU) within real-time digital signal processing (DSP) dataflow graphs represents a highly specialized challenge at the intersection of programming language runtimes, compiler theory, and low-latency systems engineering.¹ When hot-swapping an executing signal graph mid-playback, the primary engineering objective is to ensure that a newly compiled graph seamlessly inherits the continuous, time-varying execution state—including oscillator phase accumulators, physical-modeling history registers, and feedback delay-line ring buffers—of the active graph. This transition must be executed on a strict, sample-accurate basis without inducing audible transients, phase cancellations, or audio thread stalls.¹

Conventional real-time audio systems, such as visual patchers and modular DAWs, typically rely on basic signal routing abstractions to modify graphs. For example, standard implementations of node recycling or graph topology manipulation only manage external input/output port connections and user-facing parameters. They lack any native awareness of, or mechanisms to preserve, the deep, time-varying internal state history of the nodes being replaced.

Consequently, hot-swapping an active delay or filter node in these standard systems causes the new node to start with zeroed history buffers, destroying delay tails, disrupting feedback loops, and producing prominent audible clicks.

To overcome these limitations, advanced compiler runtimes utilize specialized state correspondence matching, zero-allocation schema evolution, and lock-free memory mutations to execute seamless, state-preserving transitions under sub-millisecond execution constraints.¹

State-of-the-Art Literature & Systems Review

The academic and industrial landscape of dynamic software updating and live-code execution contains several distinct methodologies for state preservation and thread safety. These range from type-driven functional reactive programming (FRP) runtimes to statically compiled, LLVM-backed live-coding machines.

Type-Driven FRP and Automatic State Migration

In the domain of functional reactive programming, research on the *Essence of Live Coding* introduces a type-safe, type-driven framework for hot-swapping executing programs in Haskell.⁵ Under this model, an active program is represented abstractly as an existential state

type ^s combined with an effectful state transition step function ⁶:

$$\text{cellStep} :: s \rightarrow a \rightarrow m(b, s)$$

The runtime environment maintains the program's active state in an isolated pointer.⁶ When the source code is modified, GHCi or a specialized compiler automatically derives a generic, type-safe migration function⁵:

$$\text{migrate} :: s_{\text{New}} \rightarrow s_{\text{Old}} \rightarrow s_{\text{New}}$$

This migration function utilizes algebraic data type reflections to recursively align identical record fields, constructor names, and newtype wrappers between the executing state and the newly compiled state.⁷ If a state variable (such as a feedback accumulator) is preserved in name and type, its value is directly injected into the new state representation; any newly introduced state variables are initialized using default values defined in the new program.⁶ While mathematically elegant and type-safe, this approach relies on lazy evaluation, a garbage-collected heap, and dynamic runtime reflection. This introduces non-deterministic execution overhead, making pure Haskell-based FRP engines unsuitable for direct execution inside high-priority, real-time audio threads.

Statically Compiled State-Tree Architectures

To bridge the gap between high-level live-coding abstractions and low-latency execution, the *mimum* (Minimal Musical Medium) compiler runtime utilizes the LLVM infrastructure to compile stateful DSP code into native machine instructions.⁸ *mimum* treats stateful operations as primary language constructs, utilizing temporal primitives such as delay, memory feedback registers (mem), and the self-referential keyword *self* (which exposes the previous sample's return value of the calling function).¹

```
Feedback Lowpass Filter in mimum syntax
fb = Input + fb
fb = (1.0 - fb) * input + fb * self
```

To preserve state during hot recompilation, the *mimum* compiler includes a specialized `state_tree` compilation pass. This pass statically analyzes the Abstract Syntax Tree (AST) of the

dsp function to identify all calls to stateful primitives and recursive loops. It then constructs a deterministic, tree-structured representation of the program's state memory requirements. When the user modifies the source code, a background worker thread recompiles the code, generates a new VM instance, and compares the new program's state tree against the executing VM's state tree. When the trees are isomorphic (e.g., when the user only changes static filter coefficients or constants), the runtime performs a direct, wholesale memory copy of the state block. If structural changes are detected, the compiler generates a dynamic patch sequence of memory-copy instructions that maps matching offsets from the old state layout to the new state layout. This ensures that delay tails, physical models, and reverb states persist seamlessly across compilation updates.¹

Dynamic Shared Library Swapping on Real-Time Runtimes

In environments executing native C/C++ binaries, systems like *Clive* implement DSU on platforms such as Bela or JACK by segregating the executing state from the active compiled logic.¹⁰ The *Clive* skeleton allocates a single, large, contiguous memory block (hardcoded to 64MB) that persists for the duration of the host application process.¹⁰ The actual DSP callback is compiled into a shared library (go.so) and loaded dynamically using the POSIX dynamic linking interface (dlopen, dlsym, dlclose).¹⁰

During a hot-swap, the host thread detects file modifications, recompiles the shared library, and retrieves the new callback function pointer via dlsym.¹⁰ The continuous 64MB state pointer is passed directly to the new callback function.¹⁰ To synchronize this operation without interrupting the real-time audio loop, *Clive* utilizes a volatile, atomic processing flag.¹⁰ The main thread spins until the active audio block processing window is complete, swaps the callback function pointer, and unloads the old library.¹⁰

Additionally, because the POSIX dynamic linker (ld) aggressively caches shared libraries of the same name—preventing the reloading of modified code—*Clive* implements a dual-buffered file-copy scheme.¹⁰ It alternates copying the newly compiled library to ./go.a.so and ./go.b.so on successive recompiles.¹⁰ This bypasses the dynamic linker's cache, preventing audio dropouts during the hot-swap.¹⁰

However, because *Clive* passes raw, untyped memory to the callback, it lacks any structural verification or safety guardrails.¹⁰ A change in the layout of the state structures within the shared library will cause an immediate segmentation fault or catastrophic memory corruption on the audio thread.¹⁰

Architectural Evolution Policies in Production Systems

The architectural limitations of managing deep internal state updates are highlighted by the design history of platforms like *SuperCollider*.¹² Historically, *SuperCollider* handles changes to Unit Generator (UGen) behavior by enforcing strict backwards compatibility.¹² If an error is fixed in a core UGen (such as a phase-alignment bug at initialization), the project's policy mandates that the old UGen remains unmodified.¹² Instead, an entirely new UGen class (e.g., PanAz2) is compiled and deployed, allowing existing artworks to compile and sound identical

on newer versions of the engine.¹²

This conservative approach avoids the complex task of dynamically migrating state between mismatched UGen binaries, but it introduces significant technical debt. This highlights the need for formal, automated state-migration frameworks in modern visual and live-coding languages.¹

State Migration Strategy Matrix

The trade-offs inherent in migrating time-varying DSP states are determined by three key factors: real-time execution safety, memory overhead, and the prevention of audio discontinuities. The following table systematically compares the primary architectural paradigms:

Migration Paradigm	Graph Diffing Complexity	Schema Evolution Overhead	Real-Time Safety Rating	Audio Artifact Mitigation	Cache Locality & Memory Performance
In-Place Proxification	$O(1)$ No structural searching; routes signals via static proxy wrappers. ¹³	High Requires maintaining dual routing pathways and manual state mapping. ¹³	Excellent Zero heap allocations; executed via $O(1)$ pointer updates on block boundaries.	Incomplete Applies equal-power crossfades, but the new node's history begins empty. ¹	High Maintains stable cache lines; does not shuffle or reallocate active node state.
Flat Snapshot Serialization	$O(N)$ Requires copying state variables to a flat intermediate buffer. ¹⁴	Moderate Requires custom serialization and deserialization functions. ¹⁴	Unacceptable High risk of page faults, heap allocations, and cache-miss penalties on the hot-path. ¹⁶	None Direct state overwrites can cause instantaneous phase or value jumps.	Very Poor Incurs severe cache-line invalidation due to double-copying large state buffers. ¹⁷

Structural Remapping (State-Tree Patching)	$O(N \log N)$ Computed entirely off-thread via AST diffing and tree matching.	Low Statically resolved off-thread; runtime executes flat patch sequences.	Excellent Statically sized patch sequences execute via lock-free memory copies.	Excellent Maintains historical states while applying sub-block crossfades. ¹	Excellent Moves data via sequential, block-aligned memory copies to minimize cache misses.
---	--	---	---	---	--

In-Place Proxification

This paradigm avoids moving state memory altogether.¹³ Instead, it instantiates the newly compiled node alongside the active node, routing the input signal to both simultaneously.¹³ An equal-power crossfade is then applied to their outputs.¹³

While this approach guarantees real-time safety, it is fundamentally incapable of preserving deep internal state. For state-heavy processors like physical-modeling waveguides, long feedback delays, or multi-tap reverberators, the new node's history buffers start completely empty. During the crossfade, the sound of the old node decays, but the new node's output is devoid of prior acoustic material, resulting in a temporary loss of echo tails or resonant density.

Flat Snapshot Serialization

In this approach, the executing node serializes its active state variables into an intermediate packed byte array.¹⁴ This array is then passed to the newly compiled node, which deserializes the data into its own memory structure.¹⁴

Under real-time constraints, standard heap allocations (malloc/free) must be strictly prohibited.¹⁷ However, even with pre-allocated memory pools, copying a large state buffer (such as a 2-second delay line at 48 kHz, requiring approximately 384 KB of float samples) twice in a single audio block cycle is highly inefficient.³ The double-copy operation invalidates CPU cache lines, creating memory bus contention and cache-miss latency spikes that can easily exceed the 1.33 ms processing window, triggering an audio dropout.¹⁶

Structural Remapping (State-Tree Patching)

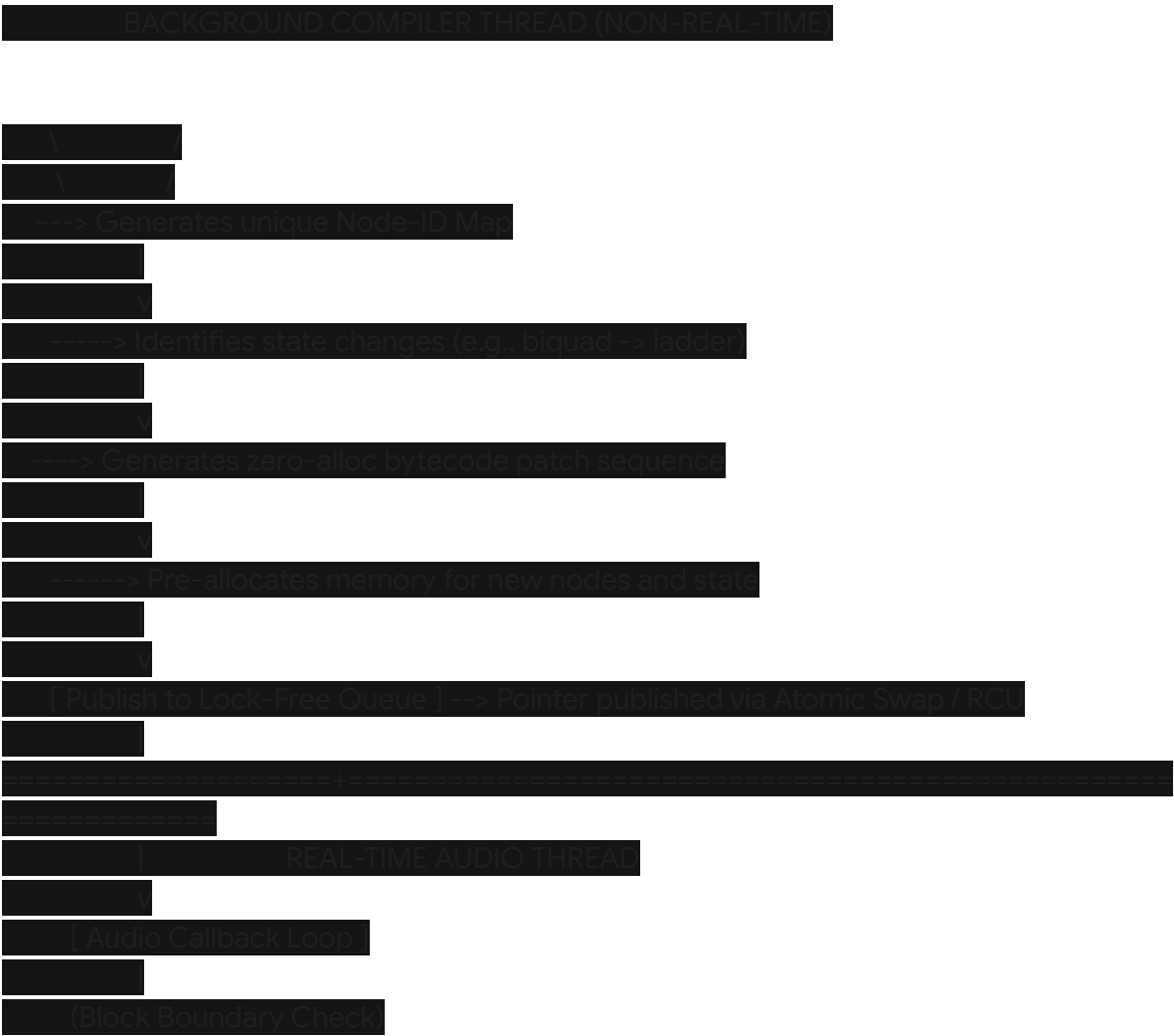
This paradigm shifts the computational complexity of state matching and layout calculation entirely to a non-real-time compiler thread. The background compiler compares the AST structures of the old and new graphs, mapping matching state variables and calculating their exact memory offsets.

On the real-time thread, this reduces the update process to executing a pre-calculated

sequence of block-aligned memory copies (memcpy). Unchanged nodes are kept in-place with their pointers intact, while updated nodes have their history buffers directly copied to their new memory locations. This combines optimal cache locality and minimal CPU overhead with complete state preservation across the hot-swap.

Architectural Blueprint for Real-Time State Transfer

This section provides a detailed architectural blueprint for implementing a lock-free, zero-allocation state transfer protocol between mismatched or evolving node architectures inside a live visual programming runtime. The proposed architecture is designed to complete all state transformations and pointer swaps within a single inter-block interval, avoiding audio thread stalls and discontinuities.³





Phase 1: AST Structural Parsing and State-Tree Isomorphism

To map state variables reliably across compilation cycles, the compiler must propagate unique identifiers through the Abstract Syntax Tree (AST). The visual program or text compiler assigns a globally unique, deterministic path-identifier to every stateful node based on its position in the dataflow graph and its lexical scope.

Let T_{old} and T_{new} represent the state-tree structures of the old and new DSP graphs, where each vertex $v \in T$ corresponds to a stateful node. The background compiler executes a tree isomorphism and diffing algorithm to compute a mapping function :

$$M : V(T_{\text{old}}) \rightarrow V(T_{\text{new}})$$

Isomorphic Matches

If a node $v_{\text{new}} \in T_{\text{new}}$ shares an identical path-identifier and type signature with a node $v_{\text{old}} \in T_{\text{old}}$, it is classified as an isomorphic match. The state memory address of the old node is preserved, or its raw byte content is marked for direct, block-aligned copying.

Structural Shifts

If a subgraph is reordered or relocated but its internal node topology remains intact, the algorithm matches the nodes using their path-identifiers and propagates the state mapping across the structural transformation.

Mismatched Nodes

Nodes present in T_{old} but absent in T_{new} are marked for reclamation. Nodes present in T_{new} but absent in T_{old} are marked for initialization.

Phase 2: Out-of-Thread Memory Layout Generation and Patch Construction

To ensure real-time safety, the audio thread must never perform structural searches or compute mappings.¹⁷ All graph allocation, code compilation, and layout comparisons are strictly segregated to a low-priority background worker thread.

The background thread executes the following sequence:

1. Allocates a contiguous block of state memory for the new graph within a pre-allocated real-time safe memory arena.
2. For each isomorphic match, the background thread calculates the source memory offset $\text{Offset}_{\text{old}}$ and destination memory offset $\text{Offset}_{\text{new}}$.
3. Constructs a flat, static array of byte-copy instructions (the *Patch Sequence*) represented as:

$$\text{Instruction} = \{\text{SrcAddr}, \text{DstAddr}, \text{ByteLength}\}$$

4. For nodes requiring initialization, the background thread pre-populates their memory offsets with default values (e.g., setting filter states to 0, or initializing phase accumulators).⁶

Phase 3: Schema Transformer Compilation (Zero-Allocation Type Coercion)

When a node's internal state structure changes, a standard byte copy is mathematically invalid. For instance, upgrading a 2-pole biquad filter to a 4-pole ladder filter, or dynamically resizing a delay-line ring buffer. To resolve this, the compiler generates a specialized, non-allocating transformation function on the background thread.¹⁵

Filter State Migration (2-Pole Biquad to 4-Pole Ladder)

A discrete-time linear time-invariant (LTI) filter is represented in state-space form as:

$$\mathbf{x}[k + 1] = \mathbf{A}\mathbf{x}[k] + \mathbf{B}u[k]$$

$$y[k] = \mathbf{C}\mathbf{x}[k] + \mathbf{D}u[k]$$

Where $\mathbf{x}[k]$ is the internal state vector (feedback/history registers).²⁰ When upgrading from a

2-pole biquad to a 4-pole ladder filter, the state vector must expand from $\mathbf{x}_{\text{old}} \in \mathbb{R}^2$ to $\mathbf{x}_{\text{new}} \in \mathbb{R}^4$. The background compiler generates a custom, zero-allocation transition kernel that maps the historical states:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}}, \quad \mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}}, \quad \mathbf{x}_{\text{new}} = 0, \quad \mathbf{x}_{\text{new}} = 0$$

Delay-Line Buffer Resizing

When a delay line is resized from length N to M , a direct memory copy of the ring buffer will misalign the active read/write playheads, causing pitch-shifting artifacts or phase cancellation. The compiler constructs a transformer that:

1. Copies the continuous history buffer from the old delay line to the new delay line.
2. Adjusts the write pointer relative to the new buffer length to ensure that the temporal delay Δt remains sample-accurate:

$$\text{WritePointer}_{\text{new}} = (\text{ReadPointer}_{\text{new}} + \Delta t \cdot f_s) \pmod{M}$$

These compiled transformer functions are appended directly to the Patch Sequence as function pointers with pre-bound memory addresses, eliminating the need for dynamic routing on the audio thread.

Phase 4: Block-Boundary Quiescent-State-Based Reclamation

To execute the hot-swap without blocking the audio thread, the runtime implements a Read-Copy-Update (RCU) pattern using atomic pointer swaps⁴:

```
C++
struct DSPGraphContainer {
    DSPGraphContainer() {}
    ~DSPGraphContainer() {}

    // Global atomically swappable pointer container
    std::atomic<DSPGraphContainer*> globalContainer = nullptr;
```

Rather than using Epoch-Based Reclamation (EBR), which requires entering and exiting critical sections around every individual node execution on the audio thread, this architecture utilizes **Quiescent-State-Based Reclamation (QSBR)**.²¹

Because real-time audio processing operates on discrete blocks, the end of the audio callback function represents a natural, global quiescent state where the audio thread holds zero references to the active DSP graph.³ This allows the system to avoid the read-side synchronization overhead of EBR.²¹

```
C++
// Real-time audio callback execution
void audio_callback(float* input_buffer, float* output_buffer, uint32_t num_samples)
    // 1. Check for a newly published graph at the block boundary
    DSPGraphContainer* new_container = g_spsc_queue.pop()
    if (new_container != nullptr)
        // Atomic exchange of active graph pointer
        DSPGraphContainer* old_container = g_graph_container.exchange(new_container,
std::memory_order_release)

    // Execute pre-compiled Patch Sequence (memcpy operations and transition kernels)
    execute_patch_sequence(new_container->patch_sequence)

    // Push the old graph to the retirement queue
    g_retirement_queue.push(old_container)

    // 2. Execute the active DSP graph
    DSPGraphContainer* current_graph =
g_graph_container.load(std::memory_order_acquire)
    current_graph->active_graph->process(input_buffer, output_buffer, num_samples)

    // 3. Signal quiescent state to the QSBR manager
    // At the end of the callback, the audio thread holds no references to the graph
    signal_quiescent_state()

```

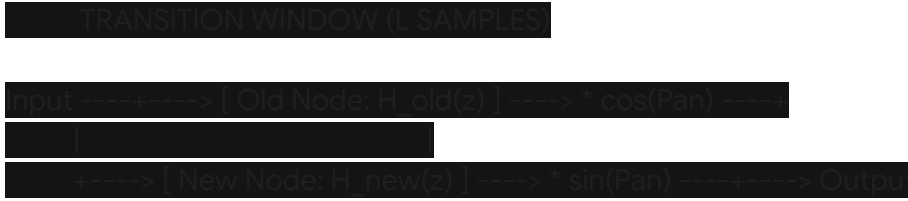
On the non-real-time thread, the QSBR manager monitors quiescent state signals.²¹ Once the

audio thread completes its callback block, the old graph container is safely deallocated on the background thread, guaranteeing zero heap deallocations (free) on the hot-path.¹⁷

Phase 5: Sub-Block Dual-Execution and Equal-Power Crossfading

Even with precise state mapping, swapping a DSP algorithm can cause immediate phase steps or transient spikes in the output signal if the mathematical transfer functions of the old and new nodes differ significantly (e.g., swapping a lowpass filter to a highpass filter).¹

To eliminate these audible discontinuities, the active graph container supports a *Dual-Execution Phase*¹³:



1. **Parallel Execution:** During the transition window of length L samples (typically spanning one or two audio blocks, e.g., 64 to 128 samples), the audio thread executes both the old node $H_{old}(z)$ and the new node $H_{new}(z)$ in parallel¹³:

$$y_{old}[t] = H_{old}(x[t]), \quad y_{new}[t] = H_{new}(x[t])$$

2. **Equal-Power Cosine/Sine Crossfade:** To maintain constant root-mean-square (RMS) energy and prevent audible volume dips at the midpoint, the outputs are mixed using an equal-power crossfade curve¹⁸:

$$y[t] = y_{old}[t] \cdot \cos\left(\frac{\pi}{2} \cdot \frac{t}{L}\right) + y_{new}[t] \cdot \sin\left(\frac{\pi}{2} \cdot \frac{t}{L}\right), \quad 0 \leq t \leq L$$

This formulation ensures that:

$$\cos^2(\theta) + \sin^2(\theta) = 1.0$$

This preserves output power for uncorrelated signals.¹⁸

3. **Phase-Aligned Transition:** For phase-sensitive signals, the transition can optionally utilize a density-aligned search or transient estimation (similar to parallel sigma-delta trellis stitching).²⁴ This matches the phase alignments of the old and new nodes before executing the swap, minimizing phase cancellation during the crossfade.²⁴
4. **Final Reclamation:** Once $t \geq L$, the crossfade is complete.¹⁸ The old node is marked as inactive, its execution path is bypassed, and its memory is retired via the lock-free epoch reclamation pipeline.¹³

Conclusions and Engineering Recommendations

Implementing dynamic software updating in real-time DSP dataflow graphs requires a strict division of labor between compile-time static analysis and runtime execution. To achieve sample-accurate state preservation without compromising real-time safety, audio software architects must adhere to several core principles:

1. **Reject Dynamic Introspection on the Hot-Path:** Systems that rely on runtime reflection, dynamic type checking, or existential type unwrapping during the audio callback are fundamentally unsafe for low-latency audio.¹⁶ Architects should design compilers that compute memory offset deltas and generate byte-copy patches *statically* on background worker threads.
2. **Utilize State-Tree Isomorphism:** By representing the DSP graph's memory requirements as a deterministic state tree, compilers can reliably track time-varying variables (such as delay lines and filter histories) across complex topological changes. This approach preserves musical continuity, allowing reverb tails and delay lines to decay naturally across code updates.¹
3. **Enforce Block-Boundary Atomic Swaps:** Lock-free, zero-allocation state migration must be synchronized with the natural boundaries of the audio processing block.¹⁰ By combining $O(1)$ atomic pointer exchanges with Quiescent-State-Based Reclamation, the real-time thread can safely load new DSP graphs and execute state patches without incurring the execution overhead or safety risks of mutexes and system-level locks.¹⁷
4. **Integrate Sub-Block Energy-Preserving Crossfades:** State matching alone cannot prevent transient pops if the underlying DSP transfer function changes abruptly.¹ Standardizing on sample-accurate, dual-execution equal-power crossfades during transition blocks is essential to guarantee seamless, artifact-free audio hot-swapping under any production conditions.¹⁸

Works cited

1. Live Coding | mimium, accessed May 23, 2026, https://mimium.org/en/docs/user_guide/livecoding/
2. Dynamic software updating - Wikipedia, accessed May 23, 2026, https://en.wikipedia.org/wiki/Dynamic_software Updating
3. Audio DSP System Overview - Soundskrit, accessed May 23, 2026, <https://soundskrit.ca/2024/06/05/audio-dsp-system/>
4. forge-audio: A lock-free audio substrate using deterministic rollback for PDC (plus a CUDA DSP offloader) : r/rust - Reddit, accessed May 23, 2026, https://www.reddit.com/r/rust/comments/1t1e8gg/forgeaudio_a_lockfree_audio_substrate_using/
5. The Essence of Live Coding: Change the Program, Keep the State! - Manuel Bärenz, accessed May 23, 2026, <https://www.manuelbaerenz.de/essence-of-live-coding/EssenceOfLiveCoding.pdf>
6. The essence of live coding - Manuel Bärenz, accessed May 23, 2026, <https://www.manuelbaerenz.de/essence-of-live-coding/EssenceOfLiveCodingAbstract.pdf>
7. The essence of live coding - Manuel Bärenz, accessed May 23, 2026, <https://www.manuelbaerenz.de/essence-of-live-coding/EssenceOfLiveCodingPresentation.html>
8. Mimium: A Self-Extensible Programming Language for Sound and Music - Academia.edu, accessed May 23, 2026, https://www.academia.edu/97545081/Mimium_A_Self_Extensible_Programming_Language_for_Sound_and_Music
9. mimium-web - Lib.rs, accessed May 23, 2026, <https://lib.rs/crates/mimium-web>
10. Clive :: mathr, accessed May 23, 2026, <https://mathr.co.uk/web/clive.html>
11. Live Coding & Hot Code Swapping– Updating a running program without restarting it - DEV Community, accessed May 23, 2026, https://dev.to/vaibhavee_singh89/live-coding-hot-code-swapping-updating-a-running-program-without-restarting-it-35f9
12. Decide on a policy for handling breaking changes when fixing UGen bugs #4976 - GitHub, accessed May 23, 2026, <https://github.com/supercollider/supercollider/issues/4976>
13. UNIVERSIDADE CATÓLICA PORTUGUESA DESIGNING SOUND: PROCEDURAL AUDIO RESEARCH BASED ON, accessed May 23, 2026, <https://repositorio.ucp.pt/bitstream/10400.14/13552/1/farnell-thesis-final.pdf>
14. State Transfer for Clear and Efficient Runtime Upgrades - Department of Computer Science, accessed May 23, 2026, <https://www.cs.tufts.edu/~jfooster/papers/ekiden.pdf>
15. Practical Dynamic Software Updating for C - Computer Science and Engineering, accessed May 23, 2026, <https://www.cs.ucr.edu/~neamtui/pubs/pldi06neamtui.pdf>
16. Trying to understand the general scheme of how real-time audio signal processing works on embedded system, accessed May 23, 2026, <https://dsp.stackexchange.com/questions/91558/trying-to-understand-the-gener>

- [al-scheme-of-how-real-time-audio-signal-processing](#)
17. Read-Copy-Update (RCU): the Secret to Lock-Free Performance - InfoQ, accessed May 23, 2026, <https://www.infoq.com/articles/read-copy-update/>
 18. Equal power crossfade - Signal Processing Stack Exchange, accessed May 23, 2026, <https://dsp.stackexchange.com/questions/14754/equal-power-crossfade>
 19. Automating Live Update for Generic Server Programs - vusec, accessed May 23, 2026, <https://download.vusec.net/papers/tse-2016.pdf>
 20. DSP Basics - Spin Semiconductor, accessed May 23, 2026, http://www.spinsemi.com/knowledge_base/dsp_basics.html
 21. Quiescent State Based Reclamation vs Epoch Based Reclamation - Stack Overflow, accessed May 23, 2026, <https://stackoverflow.com/questions/36573370/quiescent-state-based-reclamation-vs-epoch-based-reclamation>
 22. I'm confused about equal power crossfading - DSP and Plugin Development Forum, accessed May 23, 2026, <https://www.kvraudio.com/forum/viewtopic.php?t=347151>
 23. Audio Editor: Nodes - Arma Reforger - Bohemia Interactive Community Wiki, accessed May 23, 2026, https://community.bistudio.com/wiki/Arma_Reforger:Audio_Editor:_Nodes
 24. mann1x/foo_dsd_trellis: foobar2000 DSD Trellis SDM DSP plugin — C/MSVC/Windows - GitHub, accessed May 23, 2026, https://github.com/mann1x/foo_dsd_trellis
 25. Quick Sampler waveform display in Logic Pro for Mac - Apple Support (TM), accessed May 23, 2026, <https://support.apple.com/en-tm/guide/logicpro/lgcp4492eed9/mac>
 26. Lock-free swap of two unique_ptr